

React JS Hands-on Lab

React Forms Validation – Mail Register App

Objective

Create a React App named **mailregisterapp** with a component named **register.js**. The form accepts name, email and password, and validates the fields as follows:

Field	Validation Rule
Name	Should have at least 5 characters
Email	Should have @ and .
Password	Should have at least 8 characters

The validations are implemented through the change event handler and the submit event handler of the form.

In this lab I have learnt how to:

- Implement React forms validation
- Use various input controls like textbox, button and textarea

Step 1: Create the React Project

```
npx create-react-app mailregisterapp
cd mailregisterapp
npm start
```

Step 2: register.js – State and Change Handler

Created the **Register** component with state for the form fields and their errors. The **handleChange** event handler validates each field as the user types, using a switch on the input name:

```
import React from 'react';

class Register extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      fullName: '',
      email: '',
      password: '',
      errors: {
        fullName: '',
        email: '',
        password: ''
      }
    }
  }

  handleChange = (event) => {
    event.preventDefault();
    const { name, value } = event.target;
    let errors = this.state.errors;
```

```

switch (name) {
  case 'fullName':
    errors.fullName =
      value.length < 5
        ? 'Full Name must be 5 characters long!'
        : '';
    break;

  case 'email':
    const validEmailRegex =
      RegExp(/^[^\s@]+@[^\s@]+\.[^\s@]+$/);
    errors.email =
      validEmailRegex.test(value)
        ? ''
        : 'Email is not valid!';
    break;

  case 'password':
    errors.password =
      value.length < 8
        ? 'Password must be 8 characters long!'
        : '';
    break;

  default:
    break;
}

this.setState({ errors, [name]: value });
}

```

Code: src/register.js (part 1 – state and handleChange)

Step 3: register.js – Submit Handler and Render

The **validateForm** function checks that none of the error messages are set. The **handleSubmit** event handler, triggered on form submit, shows a success alert if the form is valid, or shows each individual error message otherwise:

```

validateForm = (errors) => {
  let valid = true;
  Object.values(errors).forEach(
    (val) => val.length > 0 && (valid = false)
  );
  return valid;
}

handleSubmit = (event) => {
  event.preventDefault();
  if (this.validateForm(this.state.errors)) {
    alert('Valid Form');
  } else {
    if (this.state.errors.fullName !== '') { alert(this.state.errors.fullName) }
    if (this.state.errors.email !== '') { alert(this.state.errors.email) }
    if (this.state.errors.password !== '') { alert(this.state.errors.password) }
  }
}

render() {
  return (
    <div className='wrapper'>
      <h1>Register Here!!!</h1>
      <form onSubmit={this.handleSubmit} noValidate>

```

```

        <div className='fullName'>
          <label htmlFor='fullName'>Name: </label>
          <input type='text' name='fullName' onChange={this.handleChange} />
        </div>
        <div className='email'>
          <label htmlFor='email'>Email: </label>
          <input type='email' name='email' onChange={this.handleChange} />
        </div>
        <div className='password'>
          <label htmlFor='password'>Password: </label>
          <input type='password' name='password' onChange={this.handleChange} />
        </div>
        <div className='submit-btn'>
          <button type='submit'>Submit</button>
        </div>
      </form>
    </div>
  )
}
}

export default Register;

```

Code: src/register.js (part 2 – validateForm, handleSubmit, render)

Step 4: App.js

```

import './App.css';
import Register from './register';

function App() {
  return (
    <div className="App">
      <Register />
    </div>
  );
}

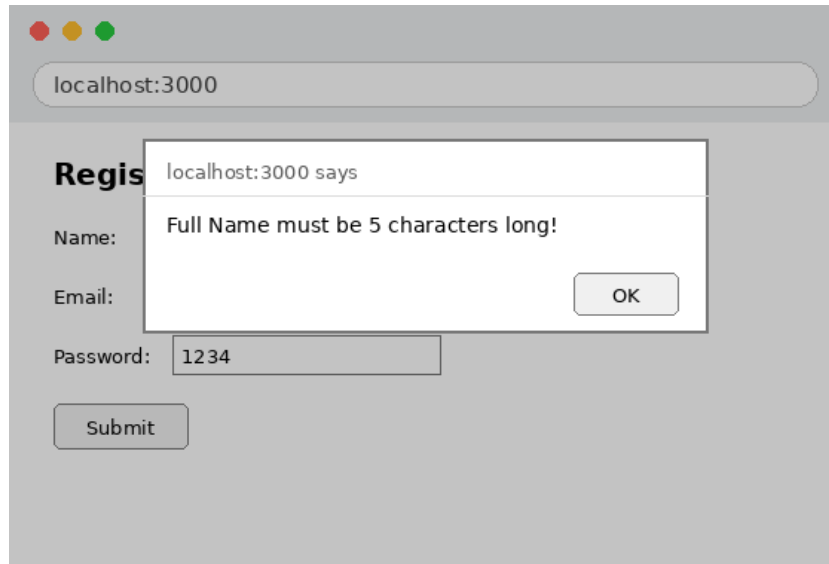
export default App;

```

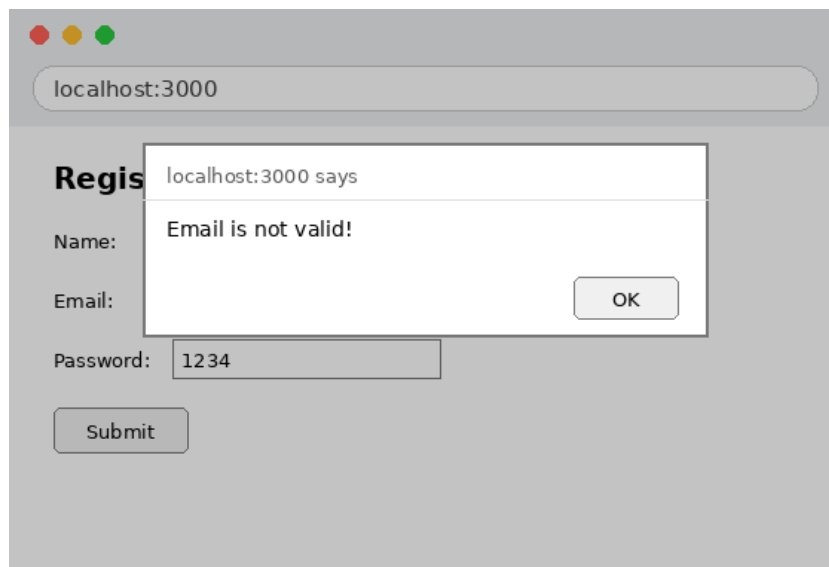
Code: src/App.js

Output Expected

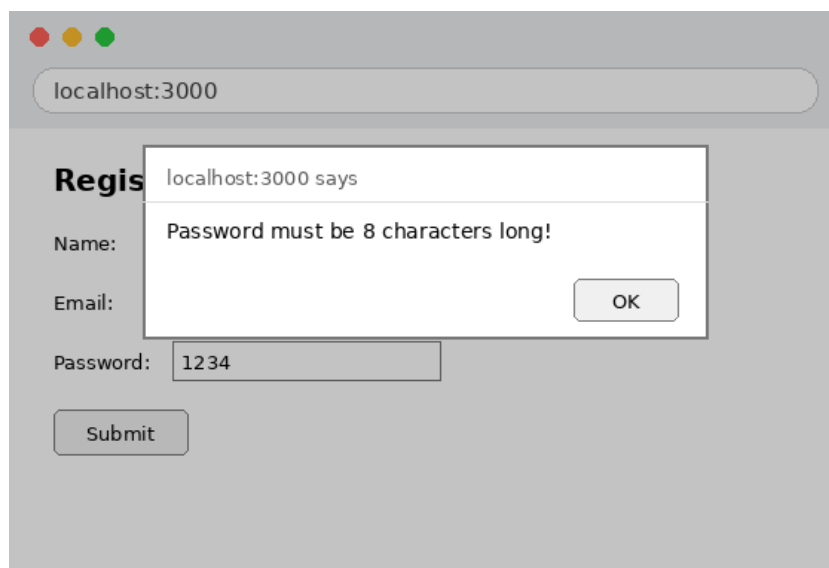
Filling the form with a short name, an invalid email and a short password and clicking Submit shows the following validation alerts one after another:



Screenshot: Alert shown when Name has fewer than 5 characters

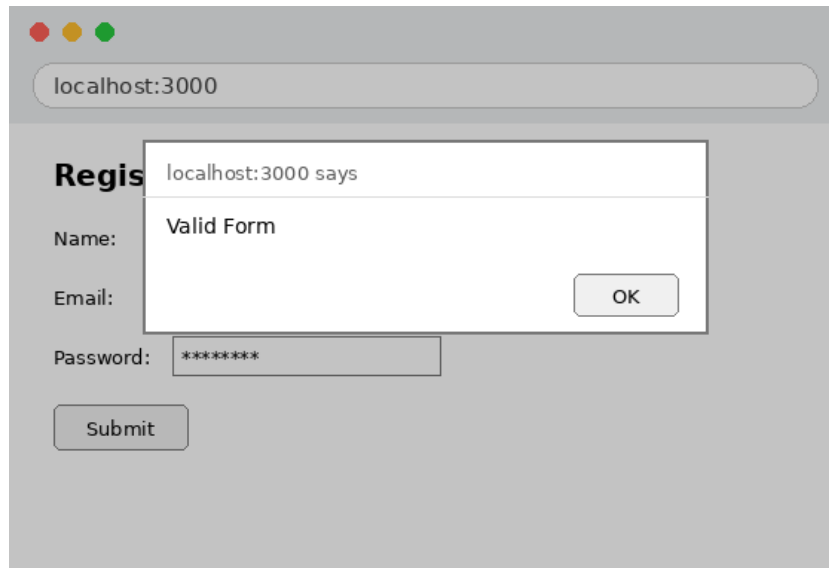


Screenshot: Alert shown when Email is not valid



Screenshot: Alert shown when Password has fewer than 8 characters

When all three fields satisfy the validation rules, submitting the form shows the success alert instead:



Screenshot: Alert shown when the form passes all validations

End of Lab